Preliminaries of Machine Learning

Hyoung Joon, Kim

March 18, 2026

Contents

Chapter 1

Introduction

This is some notes about preliminaries of machine learning, based on the book by Kevin Murphy.

Chapter 2

Probability

2.1 Basic Definitions

Definition 2.1.0.1 (Probability Space)

A *probability space* is a triple $(\Omega, \mathcal{F}, \mathbb{P})$ where

- Ω is a non-empty set called the *sample space*;
- $\mathcal{F}$ is a σ-algebra of subsets of the sample space Ω;
- $\mathbb{P} : \mathcal{F} \to [0, 1]$ is a probability measure.

2.2 Lists of Probability Distributions

2.2.1 Discrete Distributions

Name	Support	Parameters	PMF	Mean, Variance
Bernoulli	$\{0, 1\}$	$p \in \{0, 1\}$	$\mathrm{Ber}(x\|p) = p^x(1-p)^{1-x}$	$\mathbb{E}[X] = p, \mathbb{V}[X] = p(1-p)$
Binomial	$\{0, 1, 2, \ldots, N\}$	$N, p \in [0, 1]$	$\mathrm{Bin}(x\|N, p) = \binom{N}{x} p^x (1-p)^{N-x}$	$\mathbb{E}[X] = Np, \mathbb{V}[X] = Np(1-p)$
Categorical	$\{0, 1, 2, \ldots, N\}$	$\boldsymbol{\theta}$	$\mathrm{Cat}(x\|\boldsymbol{\theta}) = \prod_{k=1}^{K} \theta_k^{\mathbb{I}(x=k)}$	
Multinomial	$\boldsymbol{x}$ where $\sum_{k=1}^{K} x_k = N$	$N, \boldsymbol{\theta}$	$\mathrm{Mult}(\boldsymbol{x}\|\boldsymbol{\theta}) = \binom{N}{x_1, x_2, \ldots, x_K} \prod_{k=1}^{K} \theta_k^{x_k}$	
Poisson	$\{0, 1, 2, \ldots\}$	$\lambda > 0$	$\mathrm{Pois}(x\|\lambda) = e^{-\lambda} \dfrac{\lambda^x}{x!}$	$\mathbb{E}[X] = \lambda, \mathbb{V}[X] = \lambda$

2.2.2 Continuous Distributions

Name	Support	Parameters	PDF	Mean, Variance
Gaussian	$\mathbb{R}$	μ, σ	$\mathcal{N}(\mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma}} e^{-\frac{1}{2\sigma^2}(x-\mu)^2}$	$\mathbb{E}[X] = \mu, \mathbb{V}[X] = \sigma^2$
Student t-distribution	$\mathbb{R}$	μ, σ	$\mathcal{T}_\nu(x\|\mu, \sigma^2) = \frac{\Gamma(\frac{\nu+1}{2})}{\sqrt{\pi\nu}\Gamma(\frac{\nu}{2})} \left(1 + \frac{x^2}{\nu}\right)^{-\frac{\nu+1}{2}}$	$\mathbb{E}[X] = 0$ for $\nu > 0$, $\mathbb{V}[X] = \frac{\nu}{\nu-2}$ for $\nu > 2$
Cauchy	$\mathbb{R}$	μ, λ	$\mathcal{C}(x\|\mu, \lambda) = \frac{1}{\lambda\pi(\frac{x-\mu}{\lambda})^2}$	undefined
Laplace	$\mathbb{R}$	μ, b	$\text{Laplace}(x\|\mu, b) = \frac{1}{2b} \exp\left(-\frac{\|x-\mu\|}{b}\right)$	$\mathbb{E}[X] = \mu, \mathbb{V}[X] = 2b^2$
Gamma	$\mathbb{R}^+$	$a, b > 0$	$\text{Ga}(x\|\text{shape} = a, \text{scale} = b) = \frac{b^a}{\Gamma(a)} x^{a-1} e^{-xb}$	$\mathbb{E}[X] = ab, \mathbb{V}[X] = ab^2$
Exponential	$\mathbb{R}^+$	λ	$\text{Expon}(x\|\lambda) = \lambda e^{-\lambda x}$	$\mathbb{E}[X] = \lambda, \mathbb{V}[X] = \lambda^2$
Chi-squared	$\mathbb{R}^+$	none	$\chi^2_\nu(x) = \text{Ga}(x\|\text{shape} = \nu/2, \text{scale} = 2)$	$\mathbb{E}[X] = \nu, \mathbb{V}[X] = 2\nu$

Chapter 3

Statistics

This part is about guessing the parameters of our probability models shown in the previous section. Most of the estimation will take the form of $\hat{\boldsymbol{\theta}} = \underset{\boldsymbol{\theta}}{\operatorname{argmin}}\ \mathcal{L}(\boldsymbol{\theta})$, where the $\boldsymbol{\theta}$ is our parameter to be estimated, and the $\mathcal{L}$ is the *loss function*.

There will be two main approaches, namely **frequentist statistics** and **Bayesian statistics**. Frequentist method treats parameters $\boldsymbol{\theta}$ as fixed value, and the data $\mathcal{D}$ as unknown and changes. Conversely, Bayesian method treats parameters $\boldsymbol{\theta}$ as unknown, and the data $\mathcal{D}$ as fixed and known.

However, note that the model may not take any parameters; such models can be adjusted by using **nonparametric methods**.

3.1 Bayesian Statistics

Bayesian statistics treats parameters $\boldsymbol{\theta}$ as unknown, and the data $\mathcal{D}$ as fixed and known. We use Bayes rule to update our uncertainty about parameters when the data is given;

$$p(\boldsymbol{\theta}|\mathcal{D}) = \frac{p(\boldsymbol{\theta})p(\mathcal{D}|\boldsymbol{\theta})}{p(\mathcal{D})} = \frac{p(\boldsymbol{\theta})p(\mathcal{D}|\boldsymbol{\theta})}{\int p(\boldsymbol{\theta}')p(\mathcal{D}|\boldsymbol{\theta}')d\boldsymbol{\theta}'}$$

Here, $p(\boldsymbol{\theta})$ is called the **prior**, and $p(\mathcal{D}|\boldsymbol{\theta})$ is called the **likelihood**. $p(\boldsymbol{\theta}|\mathcal{D})$ is called the **posterior**, and $p(\mathcal{D})$ is called the **marginal likelihood**.

After the calculation of posterior distribution, we can try to predict the future observations by calculating the **posterior predictive distribution**:

$$p(x|\mathcal{D}) = \int p(x|\boldsymbol{\theta})p(\boldsymbol{\theta}|\mathcal{D})d\boldsymbol{\theta}$$

We are marginalizing out all the unknown parameters. If the calculation is too expensive, we can just plug-in the paramter value we estimated, like MLE or MAP.

$$p(x|\mathcal{D}) \approx p(x|\hat{\boldsymbol{\theta}})$$

This is called **plug-in approximation**.

3.1.1 Selecting the Prior

Selecting appropriate prior for parameters are one of the challenging problem of Bayesian method. There are several choices;

- conjugate prior
- uninformative prior

- hierarchical prior
- empirical prior

3.2 Frequentist Statistics

Frequentist statistics assumes that there is a true parameter value $\boldsymbol{\theta}$ and tries to estimate the parameter value by changing the data. The change of data will change the quantity which is used to estimate the parameter, and we use this change to check which is the best estimation.

Even if the frequentist method has some drawbacks and even paradoxes, several concepts such as cross validation is useful even in the Bayesian method. Here, the basic concepts are shown.

3.2.1 Sampling Distribution

3.2.2 Fisher Information Matrix

3.3 Conjugate Priors

Calculating the posterior distribution can be untractable, if the prior and likelihood does not match well. Here the well-matching prior according to the likelihood is given, called the **conjugate priors**.

likelihood	**unknown parameters**	**conjugate prior**	**parameters(prior)**	**posterior**

3.4 Uninformative Priors

3.5 Hierarchical Priors

3.6 Empirical Priors

Chapter 4

Decision Theory

After

Chapter 5

Graphical Models

Chapter 6

Information Theory

Chapter 7

Optimization

```
# sample code block
def is_prime(n):
    if n < 2:
        return false
    for i in range(n**0.5, 2):
        if n % i == 0
            return false

    return true
```